Deliverable 3 - INF201

Submission deadline: 13th Nov 2026

Hand-in information:

- This is a group submission, so one person submits on behalf of the pair/group
- Submit Python scripts (.py), a Jupyter notebook (ipynb) *and* a pdf of the same notebook
  - See Overview and objectives for a list of what your notebook should contain
- Include a link to your GitHub repository and a log of the commits
  - To generate the log, paste this into the terminal after your last commit: `git log --graph --pretty=format:"%h - %an, %ar : %s" > git_log.txt`
  - The repository should contain a `requirements.txt` with information on your virtual environment
  - Note: Remember to create a new virtual environment for each deliverable
- Include a discussion of GenAI use, for example a summary of the chat log as you have done earlier
  - This can be a separate file or be included in the Jupyter notebook
- Make a short (5 mins) video discussing your code and results
  - Note that you both have to contribute to the video
  - Either by recording it together
  - Or recording parts separately and editing it together
- We would appreciate it if use English for this assignment, both in the writing and in the video

Overview and objectives

In this assignment you will continue working on the Character class in `game_simulator_v1.py` that we created in lecture_08 (check out the Week 44 module), and expand this to create a mini-RPG in the form of a battle simulator. To do this you will update and extend the existing class and methods, introduce new classes allowing us to use dice and weapons, and create a combat manager class.

The rules for the game is as follows:

- The game you build should allow two characters to battle until one of them wins.
- The characters can use weapons to attack, where the damage dealt is determined by a dice roll.
- The characters can have different specialisations (warrior or mage).
- A character's HP (hit points, or health) must be between 0 and the max given for this specific character.
- The character dies when it reaches 0 HP.

Learning Objectives:

- Enforce data encapsulation using private attributes and properties.
- Implement custom classes to model discrete concepts (`Dice` , `Weapon` , `Battle`).¨
- Create subclasses inheriting from the `Character` class.
- Apply object composition by equipping characters with weapons and dice.

To organise your project, we recommend the following struckture:

- Separate modules (Python scripts) for each of the classes created
- A Jupyter Notebook containing the following:
  - A presentation of each class created (showing attributes and methods objects of the class has)
  - Simulated battles between different types of characters, with and without weapons

Task 1: Update the Character class

When we created the Character class in the lecture we first included an `attack()` method, and afterwards indicated that we wanted hit points to be a protected feature `_hp` . This lead to the `Character.attack()` method changing the target's protected attribute. Modifying another object's private attribute is bad form! Your first task is to fix this.

- Implement a `take_damage()` method to be used when the `Character` object is injured. This method should only affect the characters *own* HP. follows good object-oriented design principles and keeps _hp properly encapsulated.
- Implement a `heal()` method to be used when the `Character` heals (in other words increase their hit points)
- Change the existing `attack()` method so that calls the target's `take_damage()` method rather than directly modifying the target's `_hp` .
- Add an `is_alive` property to the `Character` class. This should be read-only, and return True if `_hp` > 0 and False otherwise.

Task 2: Introducing randomness with a Dice class

Currently, characters deal fixed damage. To introduce variable damage, create a separate `Dice` class.

- Create a `Dice` class that can create any n-sided Dice (n>=2), where the default number of sides is 6.
- Add a `roll()` method to roll the dice, using `random.randint(1, self.sides)` .
- Add a `__repr__` method that returns string representations like `Dice(sides=6)` .
- Implement a `__call__` method that allow you to roll the dice created by `d20 = Dice(sides=20)` by calling `d20.roll()` .
  - See the Python documentation of `__call__` or this example

Task 3: Create items

Create a `Weapon` class in `items.py` that utilizes your `Dice` class for damage calculations. We call the module `items.py` rather than `weapons.py` so that it can be expanded to include other items your characters can hold.

- Define the class `Weapon` with the attributes `name` and `damage_dice: Dice`.
- Include a `roll_damage()` method that calls `self.damage_dice.roll()`
- Update `Character.__init__` to accept an optional weapon.
- If a character has an equipped weapon, `attack()` should roll damage from the weapon instead of using default base damage.

Task 4: Character specialization

Extend the character module to include two specialised subclasses of `Character`.

1. `Warrior` subclass (defensive specialist)
   - New attribute: `armor: int`, with a default value of 5
   - Override the `Character.take_damage()` method by reducing the incoming damage by armor before applying it.
2. `Mage` subclass (resource-based attacker)
   - New attributes: `mana: int` and `mana_max: int`, both with default value 30
   - Magic attacks consume mana, so the mana available determines whether the mage can use magic.
   - Override the `Character.attack()` method:
     - If the mage has at least 10 mana left, spend 10 mana to deal a magical spell attack dealing double damage. This magic attack does not use any weapons, even if the character has one equipped.
     - If the mage has less than 10 mana left, fall back to the base attack, and restore 5 mana. This base attack should use whichever weapon the character has equipped.

Task 5: Running battles

Create a `Battle` class to manage turn-based combat between two `Character` objects.

- Initialize `Battle` with `hero: Character` and `enemy: Character`.
- Implement a `play_round()` method where `hero` attacks `enemy`, and if `enemy` is still alive, `enemy` attacks `hero`.
- Implement a `simulate_battle()` method with a `while` loop that runs rounds until one character is no longer alive.